

Plot Data Files Guide

This directory contains JSON data files for all generated plots. You can regenerate or customize plots without re-running the full analysis pipeline.

Available Data Files

Each plot has a corresponding `*_data.json` file:

1. **checkpoint_timeline_data.json** - All checkpoints by date with mAP values
2. **trackB_checkpoints_metrics_data.json** - Top 10 checkpoints with all metrics
3. **trackB_runs_metrics_data.json** - Top 12 evaluation runs with metrics
4. **weighted_vs_unweighted_data.json** - Comparison between 0.3708 and 0.3904
5. **trackC_categories_data.json** - Track C category distributions
6. **efficiency_configs_data.json** - Efficiency configurations with latency
7. **run_timeline_data.json** - Timeline of all runs by date
8. **backbone_pretraining_data.json** - Backbone and pretraining distributions
9. **backbone_metrics_comparison_data.json** - ResNet18 vs VideoMAE comparison
10. **trackB_vs_trackC_comparison_data.json** - Track B vs Track C metrics

Usage Options

Option 1: Regenerate All Plots (Same Style)

```
python local_extraction/final_scripts/regenerate_plots_from_data.py
```

This recreates all PNG plots from the JSON data files with the same styling.

Option 2: Regenerate to Different Directory

```
python local_extraction/final_scripts/regenerate_plots_from_data.py --output-dir custom_output/
```

Option 3: Custom Analysis with Python

Load any JSON file and create custom visualizations:

```
import json
import matplotlib.pyplot as plt

# Load data
```

```

with open('efficiency_configs_data.json', 'r') as f:
    data = json.load(f)

# Custom plot
configs = data['configs']
names = [c['name'] for c in configs]
mAPs = [c['mAP'] for c in configs]
latencies = [c['latency_ms'] for c in configs]

# Create your own visualization
fig, ax = plt.subplots()
ax.scatter(latencies, mAPs)
ax.set_xlabel('Latency (ms)')
ax.set_ylabel('mAP (%)')
ax.set_title('Custom: Latency vs mAP')
plt.savefig('custom_plot.png')

```

Option 4: Load in Other Tools

The JSON files can be loaded in:

- **R:** `jsonlite::fromJSON("checkpoint_timeline_data.json")`
- **MATLAB:** `data = jsondecode(fileread('checkpoint_timeline_data.json'))`
- **Excel/Sheets:** Use online JSON to CSV converters
- **Pandas:** `pd.read_json("checkpoint_timeline_data.json")`

JSON File Structure

Each file contains:

```

{
  "plot_type": "efficiency_configs",
  "configs": [
    {
      "name": "Fr=8(uni)",
      "mAP": 37.547,
      "N_top5_mAP": 13.646,
      "latency_ms": 23.649,
      ...
    }
  ]
}

```

Benefits

- ☑ **No re-analysis needed** - Plots can be regenerated without re-running evaluation
- ☑ **Customizable** - Modify colors, fonts, sizes, or create entirely new visualizations

- ☑ **Portable** - Share data without sharing entire codebase
- ☑ **Reproducible** - Exact data used for thesis plots is preserved
- ☑ **Multi-tool** - Use in Python, R, MATLAB, or spreadsheet software

Example: Modify Plot Style

```
import json
from pathlib import Path
import matplotlib.pyplot as plt
import numpy as np

# Load data
data = json.loads(Path('weighted_vs_unweighted_data.json').read_text())

# Extract metrics
w = data['weighted']['metrics']
u = data['unweighted']['metrics']

# Custom plot with different style
metrics = list(w.keys())
w_vals = list(w.values())
u_vals = list(u.values())

x = np.arange(len(metrics))
width = 0.35

fig, ax = plt.subplots(figsize=(10, 6))

# Use different colors
ax.bar(x - width/2, w_vals, width, label='Weighted', color='darkblue',
alpha=0.9)
ax.bar(x + width/2, u_vals, width, label='Unweighted', color='orange',
alpha=0.9)

# Custom styling
ax.set_ylabel('Score (%)', fontsize=14, fontweight='bold')
ax.set_title('My Custom Comparison', fontsize=16)
ax.set_xticks(x)
ax.set_xticklabels(metrics, rotation=45, ha='right')
ax.legend(fontsize=12)
ax.grid(axis='y', alpha=0.2, linestyle='--')

plt.tight_layout()
plt.savefig('my_custom_plot.png', dpi=300)
```

Notes

- All percentage values are stored as actual percentages (e.g., 37.5 not 0.375)
- Dates are stored as ISO format strings (e.g., "2025-12-26T00:00:00")

- Color codes use hex format (e.g., "#4CAF50")
- Missing values are omitted (not stored as null/None)

Regeneration Script Location

Script: `local_extraction/final_scripts/regenerate_plots_from_data.py`

Usage:

```
# From workspace root
python local_extraction/final_scripts/regenerate_plots_from_data.py

# With custom output
python local_extraction/final_scripts/regenerate_plots_from_data.py \
  --data-dir path/to/json/files \
  --output-dir path/to/output
```